MODULE

15

Type hints & code quality

Catch bugs before you run anything, and keep a team's code easy to read.

YOU WILL LEARN

Type hint basics	`01_type_hints_basics.py`
Advanced typing	`02_advanced_typing.py`
mypy catches bugs	`03_mypy_catches_bugs.py`
ruff: lint + format	`04_ruff_and_style.py`
Generics, typed decorators, @overload	`05_generics_and_typed_decorators.py`
Interview Q&A	`INTERVIEW.md`

WHY TYPE HINTS

Bugs that hide until production

THE PROBLEM

get_user(id) sometimes returns **None**. Nobody noticed, because the test user always existed. In production: **AttributeError: 'NoneType' object has no attribute 'email'** at 3 a.m.

THE IDEA

Type hints say what a function expects and returns. A checker (mypy) reads them and finds the bug without running the code.

TRY IT TYPE HINTS

The basics in one screen

```python
from dataclasses import dataclass

@dataclass
class User:
    id: int
    email: str
    nickname: str | None = None          # may be missing

def find_user(users: list[User], user_id: int) -> User | None:
    return next((u for u in users if u.id == user_id), None)

def greeting(user: User) -> str:
    return f"Hi {user.nickname or user.email}"

users = [User(1, "v@ex.com", "Vishal")]
if (u := find_user(users, 1)) is not None:     # narrow the type
    print(greeting(u))
print(find_user.__annotations__)
```

OUTPUT

```
Hi Vishal
{'users': list[__main__.User],
'user_id': <class 'int'>, 'return':
__main__.User | None}
```

1. Hints are **not enforced** at runtime: Python ignores them.
2. **X | None** forces you to handle "not found".
3. After **is not None**, mypy knows **u** is a User.

COMMON MISTAKE MYPY

The bug mypy finds for free

✗ WRONG

```
def find_user(users, user_id):
    for u in users:
        if u.id == user_id:
            return u

user = find_user(users, 42)
print(user.email)    # crashes when not found
```

Nothing warns you. The crash appears only when someone asks for a user that doesn't exist.

✓ RIGHT

```
def find_user(users: list[User], user_id: int) -> User | None: ...

user = find_user(users, 42)
print(user.email)
# mypy: error: Item "None" of "User | None"
#       has no attribute "email"
```

Run **mypy src/** in CI. With hints, the mistake is caught at review time, not at 3 a.m.

STEP BY STEP ADVANCED TYPING

Typing tools you'll actually use

TOOL	MEANS	EXAMPLE
`Literal`	only these exact values	`Literal["asc", "desc"]`
`TypedDict`	a dict with known keys (JSON)	`class OrderJSON(TypedDict): id: int`
`Protocol`	anything with these methods	`class Notifier(Protocol): def send(...)`
`Callable[[int], str]`	a function taking int, returning str	callbacks, `key=`
`TypeVar / Generic[T]`	"same type in, same type out"	`def first(xs: list[T]) -> T`
`Final`, `ClassVar`	constant / class-level	`MAX_RETRIES: Final = 3`

TRY IT GENERICS

Write once, keep the types

```python
from typing import Generic, TypeVar

T = TypeVar("T")

class Page(Generic[T]):
    def __init__(self, items: list[T], total: int) -> None:
        self.items, self.total = items, total

    def first(self) -> T | None:
        return self.items[0] if self.items else None

orders: Page[int] = Page([1042, 1043], total=57)
names: Page[str] = Page(["vishal"], total=1)
print(orders.first(), names.first(), orders.total)
# mypy knows orders.first() is int | None
```

OUTPUT

```
1042 vishal 57
```

1. **Page[int]** and **Page[str]** share one implementation.
2. Python 3.12 shortcut: **class Page[T]:** and **def first[T](...)**.

STEP BY STEP RUFF

Code quality on autopilot

COMMAND	WHAT IT DOES
`ruff check .`	finds bugs and smells: unused imports, bare except, mutable defaults
`ruff check --fix .`	fixes the safe ones automatically
`ruff format .`	formats like Black: no more style debates in reviews
`mypy --strict src/`	type checks everything
`pre-commit`	runs them before every commit

> In a team: the **CI fails** if ruff or mypy fail. Reviews then focus on logic, not commas.

IN REAL JOBS TYPING

Types power your frameworks

FastAPI reads hints to validate

```
@app.get("/orders/{order_id}")
def read(order_id: int, q: str | None = None): ...
```

Pydantic models = typed JSON

```
class OrderIn(BaseModel):
    item: str
    qty: int = Field(gt=0)
```

SQLAlchemy 2.0 typed models

```
class User(Base):
    id: Mapped[int] = mapped_column(primary_key=True)
```

CI gate

```
- run: ruff check . && mypy src/
```

INTERVIEW SAY THIS IN THE INTERVIEW

“Are type hints enforced in Python? Why use them?”

- No: Python ignores them at runtime. They’re stored in **__annotations__**.
- Tools use them: **mypy/pyright** find bugs before running, IDEs autocomplete, and they document the code.
- Frameworks **do** use them at runtime: FastAPI and Pydantic validate and convert data from hints.
- Biggest win: **Optional** handling. **X | None** forces you to deal with ”not found”.

Follow-up they ask next: ”What’s the difference between Any and object?” (Any switches type checking off; object accepts anything but lets you do almost nothing with it until you narrow it).

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you keep code quality high in a team?”

- **Formatter** (ruff format) so style is never discussed.
- **Linter** (ruff check) and **type checker** (mypy) in **pre-commit and CI**.
- **Tests** with coverage on critical paths (Module 16).
- Small **pull requests** with code review; clear names; docstrings for public functions.
- Track tech debt; refactor in small, tested steps.

> **Follow-up they ask next:** ”What is a Protocol vs an ABC?” (structural vs nominal typing; see Module 12).

RECAP REMEMBER THESE

Module 15 in 5 lines

1. Hints describe expectations; Python doesn't enforce them, **mypy** checks them.
2. **X | None** + **is not None** narrowing kills the classic None bug.
3. **Literal**, **TypedDict**, **Protocol**, **Callable**, **Generic[T]** for precise types.
4. **ruff check** + **ruff format** + **mypy** in pre-commit and CI.
5. FastAPI, Pydantic and SQLAlchemy all run on type hints.

PRACTICE NOW

Do exercises/ex15_add_type_hints.py, then make mypy --strict pass.